Architecture Specification Document

Parameterizable Output-Stationary INT8 Systolic Array AI Accelerator

SSCS Chipathon 2026 | Track A | Slot A
PDK: GF180MCU (gf180mcu_fd_sc_mcu7t5v0, 7-track, 5V)
Target: Slot A (~1,248,806 µm²) | Target Frequency: 25 MHz

Team Name: Maxilerator
Members: Irene Raphael, Amal Kunnath Anil Narayana, Muhammed Rabin K C, Muhammad Faqih Ilmi, Akhil S Nair
University: TU Munich, NTUST, IIT Delhi
Date: 30/06/2026 | **Version:** v1.0

Section 1 — Overview and Objectives

1.1 Design Description

This document describes the architecture of a parameterizable output-stationary INT8 systolic array AI accelerator designed for submission to the SSCS Chipathon 2026 Track A. The design implements tiled matrix-matrix multiplication targeting edge TinyML inference applications. The accelerator supports matrices larger than its native 8×8 compute array through a tiling strategy with ping-pong buffered SRAM for seamless back-to-back tile processing.

1.2 Design Goals

Functional Goals

- Implement a fully functional 8×8 output-stationary systolic array
- Support INT8 signed matrix multiplication with 21-bit accumulation
- Support tiled computation for matrices larger than 8×8
- Achieve ping-pong buffered SRAM for zero-overhead tile loading
- Target operating frequency: 25 MHz on GF180MCU

Reusability Goals

- Publish accelerator_core as a standalone reusable IP on GitHub
- Provide AXI4-Stream wrapper for plug-and-play SoC integration
- Keep systolic_array and mac_unit as maximally reusable primitives
- Maintain clean separation between compute, control, and interface logic

Verification Goals

- Verify all modules using cocotb Python-based simulation
- Verify AXI wrapper through simulation with AXI VIP
- Demonstrate physical chip operation via laptop Python test script

1.3 Key Design Decisions

- Output-stationary dataflow: accumulators persist across all tiling passes
- Two sram128x8 macros in ping-pong configuration for simultaneous load and compute
- Feeder skew buffer: shifts every 16 cycles matching SRAM read period
- host_interface handles physical pin protocol for Chipathon testing only
- AXI4-Stream wrapper provided separately for IP reuse in SoCs (simulation only)
- Simple valid/ready handshake at accelerator_core boundary
- Controller is pure FSM: manages modules, never touches data path
- swap signal serves dual purpose: triggers SRAM swap AND feeder restart
- ready_in is a permission pulse: held HIGH until first byte of new tile received

Section 2 — System Architecture

2.1 Output-Stationary Dataflow

The design uses output-stationary dataflow where each MAC unit holds its accumulator stationary across all tiling passes. For an N×N matrix multiply C = A × B:

- A values enter the LEFT edge of the array and flow RIGHT through pipeline registers
- B values enter the TOP edge of the array and flow DOWN through pipeline registers
- Each MAC[i][j] accumulates A[i][k] × B[k][j] for all inner steps k across all passes
- Accumulators are NEVER reset between tile passes
- Accumulators reset ONLY after all passes complete and results are fully sent
- mac_unit output (accum_out) is combinational from accumulator FF: valid one cycle after valid pulse

2.2 Tiling Strategy

For matrices larger than ARRAY_SIZE×ARRAY_SIZE the computation is split into tiles. Each tile is an 8×8 sub-block. The host signals the final pass by asserting last_pass=1 before sending the last tile.

Example — 32×32 matrix multiply using 8×8 array (4 passes required):

```
C = A × B  where A, B, C are 32×32 matrices
Number of passes = 32 ÷ 8 = 4

Computing output tile C[0:7][0:7]:
  Pass 1 (last_pass=0): A[0:7][0:7]   × B[0:7][0:7]   → partial sum
  Pass 2 (last_pass=0): A[0:7][8:15]  × B[8:15][0:7]  → adds to accumulator
  Pass 3 (last_pass=0): A[0:7][16:23] × B[16:23][0:7] → adds to accumulator
  Pass 4 (last_pass=1): A[0:7][24:31] × B[24:31][0:7] → final sum → output

After pass 4: output 64 INT8 results, clear accumulators
```

2.3 Ping-Pong Buffering

Two sram128x8 macros operate in ping-pong configuration allowing simultaneous compute and load with zero overhead:

```
During COMPUTE_AND_LOAD:
  Active SRAM:   feeder reads tile data → feeds systolic array
  Inactive SRAM: host writes next tile → zero-overhead loading

On tile_done (single-cycle pulse from host):
  controller asserts swap=1 AND ready_in=1 SAME CYCLE
  Memory swaps at NEXT rising edge → SRAMs exchange roles
  Feeder restarts reading from new active SRAM → back-to-back tiling
```

2.4 Two Usage Scenarios

Scenario 1 — Chipathon Physical Chip

```
Laptop (Python) → USB/FTDI → PCB → physical pins
                                 → host_interface (pad translation)
                                 → accelerator_core
```

host_interface translates physical pin protocol to internal signals. Used only for Chipathon testing and validation. Not part of reusable IP.

Scenario 2 — IP Reuse in SoC

```
DMA (AXI4-Stream tile data)    → axi4_stream_slave  ┐
DMA (AXI4-Stream results)      ← axi4_stream_master ├→ accelerator_core
```

AXI wrapper translates AXI4-Stream signals to identical internal accelerator_core signals. host_interface is not used in this scenario. last_pass is carried via TUSER[0] sideband. tile_done maps to TLAST.

Section 3 — Module Hierarchy

3.1 Chipathon Submission Hierarchy

```
accelerator_top                  [ARRAY_SIZE, ACCUM_WIDTH]
  ├── host_interface             [no parameters]
  └── accelerator_core           [ARRAY_SIZE, ACCUM_WIDTH]
        ├── memory               [no parameters]
        │     ├── sram128x8 (SRAM_0)
        │     └── sram128x8 (SRAM_1)
        ├── feeder               [ARRAY_SIZE]
        ├── controller           [ARRAY_SIZE]
        ├── systolic_array       [ARRAY_SIZE, DATA_WIDTH, ACCUM_WIDTH]
        │     └── mac_unit [8×8] [DATA_WIDTH, ACCUM_WIDTH]
        └── output_processor     [ARRAY_SIZE, ACCUM_WIDTH]
```

3.2 AXI Wrapper Hierarchy (IP Reuse, Simulation Only)

```
accelerator_axi_wrapper          [ARRAY_SIZE, ACCUM_WIDTH, TDATA_WIDTH]
  ├── axi4_stream_slave          [TDATA_WIDTH]
  ├── axi4_stream_master         [TDATA_WIDTH]
  └── accelerator_core           [ARRAY_SIZE, ACCUM_WIDTH]
```

3.3 Reusability Levels

Level	Modules	Description
1 — Most Reusable	mac_unit, systolic_array	Pure arithmetic, no memory or interface dependency
2 — Core IP	accelerator_core	Full accelerator with simple handshake ports, reusable in any SoC
3 — System Specific	host_interface, accelerator_top	Chipathon tapeout only
4 — Reference Only	accelerator_axi_wrapper	SoC integration reference, simulation only, not taped out

Section 4 — Module Specifications

Each module specification includes: purpose, parameters, port declaration with descriptions, internal operation, handshaking with neighboring modules, and key RTL implementation notes.

4.1 accelerator_top

Purpose

Top-level Chipathon submission wrapper. Purely structural — instantiates host_interface and accelerator_core and wires them together. Contains no logic of its own. This is what gets synthesized, placed, routed, and taped out on the GF180MCU die.

Parameters

Parameter	Default	Description
ARRAY_SIZE	8	Systolic array dimension, propagated to accelerator_core
ACCUM_WIDTH	21	Accumulator bit width, propagated to accelerator_core

Port Declaration

```
module accelerator_top #(
  parameter ARRAY_SIZE  = 8,
  parameter ACCUM_WIDTH = 21
)(
  input  logic        clk,
  input  logic        rst_n,
  inout  logic [7:0]  pad_data,
  input  logic        pad_valid_in,
  output logic        pad_ready_in,
  input  logic        pad_tile_done,
  input  logic        pad_last_pass,
  output logic        pad_valid_out,
  input  logic        pad_ready_out
);
```

Port Descriptions

Port	Dir	Width	Connects To	Description
clk	in	1	All modules	System clock (25 MHz target)
rst_n	in	1	All modules	Active low asynchronous reset
pad_data	bidir	8	host_interface	Bidirectional data bus: tile data in / results out
pad_valid_in	in	1	host_interface	Host asserts when sending valid data byte
pad_ready_in	out	1	host_interface	Permission pulse: held HIGH until first byte of new tile received

Port	Dir	Width	Connects To	Description
pad_tile_done	in	1	host_interface	Single-cycle pulse: all 128 bytes of tile transferred
pad_last_pass	in	1	host_interface	Host asserts before/during final tile pass
pad_valid_out	out	1	host_interface	Chip asserts when result byte is valid
pad_ready_out	in	1	host_interface	Host asserts when ready to accept result byte

4.2 host_interface

Purpose

Handles all physical pin-level communication for Chipathon testing. Pure pad-to-signal translation: converts bidirectional pad_data bus into separate unidirectional data_in and data_out internal signals. Controls pad output enable (OE) for the bidirectional pad cell. Contains no compute logic. Not part of reusable IP.

Parameters

None. All behavior is fixed by the physical pin protocol and GF180MCU pad cell requirements.

Port Declaration

```
module host_interface (
  input  logic        clk,
  input  logic        rst_n,
  // Physical pad side
  inout  logic [7:0]  pad_data,
  input  logic        pad_valid_in,
  output logic        pad_ready_in,
  input  logic        pad_tile_done,
  input  logic        pad_last_pass,
  output logic        pad_valid_out,
  input  logic        pad_ready_out,
  // Internal side → accelerator_core
  output logic [7:0]  data_in,
  output logic        valid_in,
  output logic        tile_done,
  output logic        last_pass,
  output logic        ready_out,
  // Internal side ← accelerator_core
  input  logic        ready_in,
  input  logic [7:0]  data_out,
  input  logic        valid_out
);
```

Port Descriptions

Port	Dir	Connects To	Description
pad_data[7:0]	bidir	GF180MCU bidir pad cell	Shared bus: carries tile bytes from host OR result bytes to host. OE=1 when

Port	Dir	Connects To	Description
			valid_out=1 (chip driving), OE=0 otherwise (host driving).
pad_valid_in	in	Host	Host holds HIGH while sending tile data bytes
pad_ready_in	out	Host	Permission pulse driven from ready_in. Held HIGH until first byte of new tile received.
pad_tile_done	in	Host	Single-cycle pulse from host after 128th tile byte transferred
pad_last_pass	in	Host	Level: host asserts before sending final tile; stays HIGH until after tile_done
pad_valid_out	out	Host	Chip holds HIGH while sending result bytes to host
pad_ready_out	in	Host	Host holds HIGH when ready to accept next result byte
data_in[7:0]	out	memory.write_data (wire)	Tile data byte. Direct pass-through from pad_data when valid_in=1.
valid_in	out	controller	Mirrors pad_valid_in directly
tile_done	out	controller	Mirrors pad_tile_done. Single-cycle pulse.
last_pass	out	controller	Mirrors pad_last_pass. Level signal.
ready_out	out	output_processor	Mirrors pad_ready_out. Backpressure from host.
ready_in	in	controller	Permission pulse from controller. Drives pad_ready_in.
data_out[7:0]	in	output_processor	Result byte to drive onto pad_data when valid_out=1
valid_out	in	output_processor	Drives pad_valid_out. Controls pad OE direction.

Handshaking with Neighboring Modules

Neighbor	Signals Exchanged	Description
Host (physical pins)	All pad_* signals	Physical pin protocol. ready_in is permission pulse; host streams tile on HIGH, stops on LOW.
controller	data_in, valid_in, tile_done, last_pass →; ready_in ←	controller drives permission pulse; host_interface passes all host signals through

Neighbor	Signals Exchanged	Description
output_processor	data_out, valid_out ←; ready_out →	output_processor drives result bytes; host_interface passes host backpressure

4.3 accelerator_core

Purpose

Top-level reusable IP boundary. Instantiates all compute and control submodules and defines their internal wiring. Exposes a clean handshake interface compatible with both host_interface (Chipathon) and AXI wrapper (SoC). Primary IP deliverable on GitHub.

Parameters

Parameter	Default	Description
ARRAY_SIZE	8	Systolic array dimension, propagated to all relevant submodules
ACCUM_WIDTH	21	Accumulator bit width, propagated to systolic_array, mac_unit, output_processor

Port Declaration

```
module accelerator_core #(
  parameter ARRAY_SIZE  = 8,
  parameter ACCUM_WIDTH = 21
)(
  input  logic        clk,
  input  logic        rst_n,
  input  logic [7:0]  data_in,
  input  logic        valid_in,
  output logic        ready_in,
  input  logic        tile_done,
  input  logic        last_pass,
  output logic [7:0]  data_out,
  output logic        valid_out,
  input  logic        ready_out
);
```

Port Descriptions

Port	Dir	Driven By	Description
data_in[7:0]	in	host_interface / AXI slave	Tile data byte. Routed directly to memory.write_data as internal wire (no controller involvement).
valid_in	in	host_interface / AXI slave	Host has valid data byte. When valid_in=1 in LOAD states, one byte is written to SRAM. ready_in is tile-level permission only, not per-byte handshake.

Port	Dir	Driven By	Description
ready_in	out	controller	Permission pulse: held HIGH until first byte of new tile received. LOW during OUTPUT state and after last tile.
tile_done	in	host_interface / AXI TLAST	Single-cycle pulse. All 128 tile bytes transferred. Triggers swap.
last_pass	in	host_interface / AXI TUSER	Level. HIGH before and during final tile pass. Triggers drain phase in feeder.
data_out[7:0]	out	output_processor	Result byte. Valid when valid_out=1 AND ready_out=1.
valid_out	out	output_processor	Chip has valid result byte. Transfer = valid_out AND ready_out.
ready_out	in	host_interface / AXI master	Host can accept result byte. Backpressure from host.

Internal Wiring

DATA PATH (direct wires, no controller involvement):

```
data_in ──────────────────────────── memory.write_data
memory.read_data ──────────────────── feeder.sram_data
feeder.a_in[0..7] ─────────────────── systolic_array.a_in[0..7]
feeder.b_in[0..7] ─────────────────── systolic_array.b_in[0..7]
feeder.valid ──────────────────────── systolic_array.valid
systolic_array.results ────────────── output_processor.results
output_processor.data_out ─────────── data_out (boundary)
output_processor.valid_out ────────── valid_out (boundary)
ready_out (boundary) ──────────────── output_processor.ready_out
```

CONTROL PATH (through controller):

```
valid_in ──────────────────────────── controller
tile_done ─────────────────────────── controller
last_pass ─────────────────────────── controller
controller.ready_in ──────────────── ready_in (boundary)
controller.write_addr ────────────── memory.write_addr
controller.write_en ──────────────── memory.write_en
controller.swap ──────────────────── memory.swap AND feeder.swap_pulse
controller.last_pass_f ───────────── feeder.last_pass
controller.clear ─────────────────── feeder.clear AND systolic_array.clear
controller.output_en ─────────────── output_processor.output_en
feeder.drain_done ──────────────── controller
output_processor.output_done ──────── controller
```

4.4 memory

Purpose

Contains two sram128x8 hard macros operating in ping-pong configuration. Manages a write port (from controller during LOAD phases) and a read port (from feeder during COMPUTE phase). The swap signal from controller switches which SRAM is active (read) and which is inactive (write). Both SRAMs operate simultaneously during COMPUTE_AND_LOAD with no port conflict. Both SRAMs idle during OUTPUT state.

Parameters

None. Localparams: DATA_WIDTH=8, SRAM_DEPTH=128, ADDR_WIDTH=7.

Port Declaration

```
module memory (
  input  logic        clk,
  input  logic        rst_n,
  // Write port (from controller)
  input  logic [7:0]  write_data,
  input  logic [6:0]  write_addr,
  input  logic        write_en,
  // Read port (from feeder)
  input  logic [6:0]  read_addr,
  input  logic        read_en,
  output logic [7:0]  read_data,
  // Ping-pong control
  input  logic        swap
);
```

SRAM Timing (gf180mcu_fd_ip_sram__sram128x8m8wm1)

From Liberty file (tt, 25°C, 5V corner):

Parameter	Value	Description
min_period	5.88 ns	Maximum frequency ~170 MHz. Our 25 MHz target (40 ns) is well within spec.
CLK→Q delay	~4.5 ns	Q valid ~4.5 ns after rising edge. Stable 35.5 ns before next edge at 25 MHz.
Read latency	2 cycles total	Cycle N: present addr. Cycle N+1: Q valid on wire. Cycle N+2: staging captures.
Write latency	1 cycle	Data, addr, write_en presented before rising edge → written at that edge.

Pipelined Read Operation

Feeder pipelines reads by presenting next address while capturing previous data. This achieves 1 byte captured per cycle after initial 2-cycle latency:

```
Cycle N:   feeder presents read_addr=addr_X, read_en=1
Cycle N+1: Q = mem[addr_X] valid on wire (4.5ns after rising edge of N)
Cycle N+2: feeder staging FF captures Q at rising edge of N+2 ✓
```

```
Pipelined (128 bytes): addr presented cycles 1–128
                       Q valid cycles 2–129
                       staging captures cycles 3–130 ✓
```

Port Descriptions

Port	Dir	Driven By	Description
write_data[7:0]	in	data_in wire (boundary)	Tile data byte. Combinational wire directly from accelerator_core data_in.
write_addr[6:0]	in	controller	Write address (0–127). Registered FF counter in controller, increments on each valid transfer.
write_en	in	controller	COMBINATIONAL: = valid_in (in LOAD states). NOT valid_in AND ready_in. ready_in is tile-level permission only. SRAM writes at rising edge when write_en=1.
read_addr[6:0]	in	feeder	Read address from feeder internal counter.
read_en	in	feeder	HIGH when feeder reading. LOW when feeder self-stopped or in drain phase.
read_data[7:0]	out	SRAM Q output	Data byte from active SRAM. Valid 2 cycles after read_addr presented (pipelined).
swap	in	controller	Single-cycle pulse. active_sram FF flips at NEXT rising edge. Same signal also goes to feeder.swap_pulse.

Ping-Pong State Table

State	SRAM_0	SRAM_1	Write Target	Read Source
INITIAL_LOAD	host writes ←	Idle	SRAM_0	—
COMPUTE_AND_LOAD (after 1st swap)	feeder reads →	host writes ←	SRAM_1	SRAM_0
COMPUTE_AND_LOAD (after 2nd swap)	host writes ←	feeder reads →	SRAM_0	SRAM_1
OUTPUT	Idle	Idle	—	—
IDLE	Idle	Idle	—	—

4.5 feeder

Purpose

Reads tile data from active SRAM using its own internal pipelined address counter. Fills A and B staging registers. Applies skew buffer delays to produce the correct diagonal wavefront pattern. Generates single-cycle valid pulses every 16 cycles. Self-stops after 128 reads. Restarts on swap_pulse. On last_pass=1, manages full 14-cycle drain phase and asserts drain_done. Resets staging and skew registers on clear signal.

Parameters

Parameter	Default	Description
ARRAY_SIZE	8	Controls staging register count, skew depth, and drain counter

Localparams

```
localparam DATA_WIDTH   = 8;
localparam TILE_BYTES   = 2 * ARRAY_SIZE * ARRAY_SIZE;   // 128
localparam DRAIN_CYCLES = 2 * (ARRAY_SIZE - 1);          // 14
localparam ADDR_WIDTH   = 7;                             // $clog2(128)
```

Port Declaration

```
module feeder #(
  parameter ARRAY_SIZE = 8
)(
  input  logic        clk,
  input  logic        rst_n,
  // From memory
  input  logic [7:0]  sram_data,
  // To memory
  output logic [6:0]  read_addr,
  output logic        read_en,
  // Control from controller
  input  logic        swap_pulse,
  input  logic        last_pass,
  input  logic        clear,
  // To systolic_array
  output logic [7:0]  a_in [ARRAY_SIZE-1:0],
  output logic [7:0]  b_in [ARRAY_SIZE-1:0],
  output logic        valid,
  // To controller
  output logic        drain_done
);
```

Port Descriptions

Port	Dir	Connects To	Description
sram_data[7:0]	in	memory.read_data	One data byte per cycle from active SRAM. Valid 2 cycles after read_addr presented.
read_addr[6:0]	out	memory.read_addr	SRAM read address from internal pipelined counter (0–127).
read_en	out	memory.read_en	HIGH while reading. LOW after self-stop (128 reads done) and during drain.
swap_pulse	in	controller.swap	Single-cycle pulse. Resets read_counter and phase_tracker only. Restarts reading. Does NOT reset staging or skew.
last_pass	in	controller.last_pass_f	Level. When HIGH and 128 reads complete, feeder enters 14-cycle drain phase.
clear	in	controller.clear	Single-cycle pulse. Resets ALL internal state: staging, skew buffers, all counters. Same signal as systolic_array clear.
a_in[i][7:0]	out	systolic_array.a_in[i]	Skewed A data for row i. Zero during warmup and drain.
b_in[j][7:0]	out	systolic_array.b_in[j]	Skewed B data for col j. Zero during warmup and drain.
valid	out	systolic_array.valid	SINGLE-CYCLE PULSE ONLY. HIGH for exactly one clock cycle per shift (every 16 cycles). HIGH every cycle during 14-cycle drain.
drain_done	out	controller	Single-cycle pulse after 14 drain cycles complete. Only on last_pass tile.

Internal Structure

Staging Registers: A_stage[ARRAY_SIZE-1:0][7:0] and B_stage[ARRAY_SIZE-1:0][7:0]. Filled serially from SRAM using pipelined reads (2-cycle latency). A_stage[i] receives A[i][k] during cycles 1–8 of each 16-cycle block. B_stage[j] receives B[k][j] during cycles 9–16.

Skew Buffer: Row i has i chained register stages. Each stage holds value for 16 clock cycles (shifts every 16 cycles, not every clock cycle). Staging values shift into skew buffer at cycle 16 (when valid pulses).

```
A_stage[0] ──────────────────────────── a_in[0]   (0 delay)
A_stage[1] ──[D16]──────────────────── a_in[1]   (1 shift = 16 cycles)
```

```
A_stage[2] ──[D16]─[D16]──────────────── a_in[2]  (2 shifts)
...
A_stage[7] ──[D16]─[D16]─[D16]─[D16]─[D16]─[D16]─[D16]─ a_in[7]  (7 shifts)
[D16] = register stage holding for 16 clock cycles. Same for B_stage → b_in.
```

SRAM Read Sequence (per 16-cycle block, inner step k)

```
Cycles 1–8:   present A column k addresses (addr = row×ARRAY_SIZE+k, row=0→7)
Cycles 9–16:  present B row k addresses (addr = 64+k×ARRAY_SIZE+col, col=0→7)
Cycle 16:     assert valid=1 (SINGLE CYCLE PULSE ONLY) ← all staging data ready
Cycle 17:     skew buffer shifts — next inner step k+1 begins

Note on read latency (2 cycles): addr presented at cycle X
  → Q valid on wire at cycle X+1
  → staging captures at rising edge of cycle X+2
Feeder pipelines reads to maintain 1 byte captured per cycle.
```

Self-Stop and Restart Behavior

```
After 128 reads (addr_127 presented at cycle 128):
  read_en → 0 (self-stop) ✅
  Feeder waits in frozen state ✅

On swap_pulse=1 (from controller):
  read_counter → 0  (reset counter only) ✅
  phase_tracker → 0 (reset phase only) ✅
  read_en → 1       (restart reading) ✅
  staging and skew registers: PRESERVED ✅

On clear=1 (from controller, after output_done):
  ALL internal state reset: staging, skew, counters ✅
```

Drain Phase (Last Tile Only, 14 Cycles)

When last_pass=1, after 128 reads complete, feeder enters drain phase. SRAM reads stop. valid=1 asserted every clock cycle for 14 consecutive cycles. a_in and b_in driven to zero.

Part 1 — Real data exits skew buffer (drain cycles 1–7, cycles 513–519 in 4-tile example):

```
Row i finishes outputting real data at drain cycle i.
After drain cycle 7: all real data has exited skew buffer.
```

Part 2 — Zeros propagate last real data to far corner (drain cycles 8–14, cycles 520–526):

```
All zeros on a_in and b_in. valid=1 every cycle.
Last real data propagates through 7 column/row hops to MAC[7][7].
After drain cycle 14: ALL 64 MAC accum_out values are correct.

After drain cycle 14: drain_done=1 (single-cycle pulse) ✅
Total drain: 2×(ARRAY_SIZE-1) = 14 cycles ✅
```

Handshaking with Neighboring Modules

Neighbor	Signals	Description
controller	swap_pulse, last_pass, clear ←; drain_done →	Controller restarts feeder via swap_pulse; feeder signals drain complete
memory	read_addr, read_en →; sram_data ←	Feeder drives pipelined read addresses; memory returns data 2 cycles later
systolic_array	a_in[0..7], b_in[0..7], valid →	Feeder drives skewed data and single-cycle valid pulses to array edges

4.6 controller

Purpose

Pure FSM orchestrating all module operations. Generates write_addr counter and write_en for SRAM loading. write_en = valid_in (in LOAD states) — ready_in is tile-level permission only, not involved in per-byte writes. Asserts swap=1 on tile_done (sole swap condition). Simultaneously asserts ready_in=1 with swap to grant host permission for next tile. Forwards last_pass to feeder. Monitors drain_done and output_done. Asserts clear after output phase. Does not touch data path.

Parameters

Parameter	Default	Description
ARRAY_SIZE	8	Controls write_addr counter width

Localparams

```
localparam TILE_BYTES = 2 * ARRAY_SIZE * ARRAY_SIZE;   // 128
localparam ADDR_WIDTH = 7;                             // $clog2(128)
```

Port Declaration

```
module controller #(
  parameter ARRAY_SIZE = 8
)(
  input  logic        clk,
  input  logic        rst_n,
  // From accelerator_core boundary
  input  logic        valid_in,
  input  logic        tile_done,
  input  logic        last_pass,
  // To accelerator_core boundary
  output logic        ready_in,
  // To memory (write control)
  output logic [6:0]  write_addr,
  output logic        write_en,
  // To memory + feeder (same signal)
  output logic        swap,
  // To feeder
  output logic        last_pass_f,
  // To feeder + systolic_array (same signal)
  output logic        clear,
```

```
    // From feeder
    input  logic        drain_done,
    // To output_processor
    output logic        output_en,
    // From output_processor
    input  logic        output_done
);
```

Port Descriptions

Port	Dir	Connects To	Description
valid_in	in	accelerator_core boundary	When valid_in=1 in LOAD states, write_en=1 and byte is written to SRAM. write_en = valid_in (not valid_in AND ready_in).
tile_done	in	accelerator_core boundary	Single-cycle pulse. SOLE condition for swap. Triggers swap=1 AND ready_in=1 simultaneously.
last_pass	in	accelerator_core boundary	Level. Sampled at tile_done. If HIGH: no swap, no ready_in, enter drain then OUTPUT.
ready_in	out	accelerator_core boundary	Tile-level permission pulse. Held HIGH from same cycle as swap until first byte of new tile received (valid_in=1 while ready_in=1). Goes LOW after first byte. LOW during OUTPUT. Not asserted after last tile. Does NOT control per-byte writes.
write_addr[6:0]	out	memory.write_addr	Registered 7-bit counter. Increments on each valid transfer. Resets to 0 on swap.
write_en	out	memory.write_en	COMBINATIONAL: write_en = valid_in (in LOAD states). NOT valid_in AND ready_in. ready_in is tile-level permission only. SRAM writes at rising edge when write_en=1.
swap	out	memory.swap AND feeder.swap_pulse	Single-cycle pulse on tile_done (non-last pass only). Goes to both memory and feeder simultaneously.

Port	Dir	Connects To	Description
last_pass_f	out	feeder.last_pass	Direct forward of last_pass input to feeder.
clear	out	feeder.clear AND systolic_array.clear	Single-cycle pulse after output_done. Resets feeder staging/skew/counters and all MAC accumulators.
drain_done	in	feeder.drain_done	Single-cycle pulse from feeder after 14 drain cycles. Triggers transition to OUTPUT state.
output_en	out	output_processor.output_en	HIGH during OUTPUT state. Tells output_processor to serialize results.
output_done	in	output_processor.output_done	Single-cycle pulse after all ARRAY_SIZE² results sent. Triggers clear and return to IDLE.

FSM States

State	ready_in	write_en	swap	output_en	Description
IDLE	1*	0	0	0	All idle. ready_in=1 to grant first tile permission. Waiting for valid_in.
INITIAL_LOAD	0	comb	0	0	Loading first tile. ready_in=0 (permission consumed on first transfer). No compute yet.
COMPUTE_AND_LOAD	0	comb	on tile_done	0	Feeder computes from active SRAM. Host loads next tile into inactive SRAM simultaneously. ready_in=0 during transfer.
OUTPUT	0	0	0	1	Serializing 64 results. Both SRAMs idle. No new tile data accepted.

* ready_in=1 in IDLE and also simultaneously with swap pulse in COMPUTE_AND_LOAD (held until first transfer of new tile).

FSM Transitions

IDLE → INITIAL_LOAD:

```
  Condition: valid_in=1 AND ready_in=1 (first byte arrives while permission held)
  Action:    ready_in→0 (permission consumed), start write_addr counter,
write_en=valid_in

INITIAL_LOAD → COMPUTE_AND_LOAD:
  Condition: tile_done=1 AND last_pass=0
  Action:    swap=1 AND ready_in=1 (SAME CYCLE)
             write_addr reset to 0
             (ready_in→0 when first byte of next tile arrives)

COMPUTE_AND_LOAD → COMPUTE_AND_LOAD (back-to-back):
  Condition: tile_done=1 AND last_pass=0
  Action:    swap=1 AND ready_in=1 (SAME CYCLE)
             write_addr reset to 0

COMPUTE_AND_LOAD → OUTPUT:
  Condition: drain_done=1 (from feeder, last_pass tile only)
  Action:    output_en=1

OUTPUT → IDLE:
  Condition: output_done=1
  Action:    clear=1 (single-cycle pulse)
             output_en=0
             ready_in=1 (permission for new computation)
```

Key Timing: swap AND ready_in Same Cycle

```
Cycle 128: tile_done=1 (host done), last_pass=0
           write_en=1 (last byte written to inactive SRAM at rising edge 128)

Cycle 129: swap=1 AND ready_in=1 (SAME CYCLE) ← controller FF output
           write_addr=0 (reset for next tile)
           Memory will swap at rising edge of cycle 130
           AXI host: can assert valid_in=1 at cycle 130 (reacts 1 cycle after TREADY)
           USB host: reacts after ms latency (swap long complete by then)

Cycle 130: Memory swapped (SRAM_1 active, SRAM_0 write target)
           Feeder restarts reading SRAM_1
           If AXI host: valid_in=1, first byte written to SRAM_0 at edge 131 ✓
           ready_in→0 after first valid transfer ✓
```

write_en and write_addr Details

```
write_en  = COMBINATIONAL = valid_in (in LOAD states only)
            SRAM captures write at same rising edge write_en=1 ✓
            No extra latency ✓

write_addr = REGISTERED (FF output)
            Stable before rising edge ✓
            Increments AFTER rising edge (at N+1) ✓
            Resets to 0 on swap ✓
```

Handshaking with Neighboring Modules

Neighbor	Signals	Description
host_interface	valid_in, tile_done, last_pass ←; ready_in →	tile_done is sole swap trigger; ready_in is permission pulse
memory	write_addr, write_en, swap →	All write control and ping-pong management
feeder	swap, last_pass_f, clear →; drain_done ←	swap restarts feeder; drain_done triggers OUTPUT
systolic_array	clear →	clear after output phase resets all accumulators
output_processor	output_en →; output_done ←	output_en starts serialization; output_done triggers clear and IDLE

4.7 systolic_array

Purpose

Pure parallel compute engine. Instantiates ARRAY_SIZE×ARRAY_SIZE mac_units. Each valid=1 pulse (single cycle) causes all 64 MACs to accumulate simultaneously. A values flow left-to-right, B values flow top-to-bottom through pipeline registers. Accumulators hold values between valid pulses. clear=1 resets all accumulators.

Parameters

Parameter	Default	Description
ARRAY_SIZE	8	Grid dimension (N×N MAC units)
DATA_WIDTH	8	Input data width (INT8)
ACCUM_WIDTH	21	Accumulator width

Port Declaration

```
module systolic_array #(
  parameter ARRAY_SIZE  = 8,
  parameter DATA_WIDTH  = 8,
  parameter ACCUM_WIDTH = 21
)(
  input  logic                          clk,
  input  logic                          rst_n,
  input  logic [DATA_WIDTH-1:0]         a_in [ARRAY_SIZE-1:0],
  input  logic [DATA_WIDTH-1:0]         b_in [ARRAY_SIZE-1:0],
  input  logic                          valid,
  input  logic                          clear,
  output logic signed [ACCUM_WIDTH-1:0]
    results [ARRAY_SIZE-1:0][ARRAY_SIZE-1:0]
);
```

Port Descriptions

Port	Dir	Driven By	Description
a_in[i][7:0]	in	feeder	Skewed A data for row i. Zero during warmup and drain. Driven every cycle.
b_in[j][7:0]	in	feeder	Skewed B data for col j. Zero during warmup and drain. Driven every cycle.
valid	in	feeder	SINGLE-CYCLE PULSE ONLY. MACs accumulate on cycles when valid=1. Every 16 cycles during SRAM reads. Every cycle during 14 drain cycles.
clear	in	controller	SINGLE-CYCLE PULSE. Resets all 64 accumulators to zero. Same signal as feeder.clear.
results[i][j]	out	mac_unit[i][j].accum_out	All 64 accumulated values. Combinational FF outputs. Valid one cycle after last valid pulse. Held until next clear.

MAC Unit Grid Connectivity

```
a_in[i] → MAC[i][0].a_in  (left edge, row i)
b_in[j] → MAC[0][j].b_in  (top edge, col j)
MAC[i][j].a_out → MAC[i][j+1].a_in  (A flows right)
MAC[i][j].b_out → MAC[i+1][j].b_in  (B flows down)
MAC[i][j].accum_out → results[i][j]
```

Output Timing

mac_unit has 1-cycle latency: accum_out is valid one cycle after valid=1. After the 14-cycle drain phase completes (drain_done=1), all 64 MAC accum_out values are guaranteed valid. The controller moves to OUTPUT state at drain_done, at which point output_processor can immediately begin reading results.

4.8 mac_unit

Purpose

Single multiply-accumulate arithmetic primitive. On valid=1: accumulator += a_in × b_in (signed INT8). a_out and b_out registered every cycle regardless of valid (systolic pipeline flow). On clear=1: accumulator resets to zero. accum_out is combinational from accumulator FF — valid one cycle after valid pulse.

Parameters

Parameter	Default	Description
DATA_WIDTH	8	Input operand width (INT8)
ACCUM_WIDTH	21	Accumulator register width

Port Declaration

```
module mac_unit #(
  parameter DATA_WIDTH  = 8,
  parameter ACCUM_WIDTH = 21
)(
  input  logic                          clk,
  input  logic                          rst_n,
  input  logic signed [DATA_WIDTH-1:0]  a_in,
  input  logic signed [DATA_WIDTH-1:0]  b_in,
  input  logic                          valid,
  input  logic                          clear,
  output logic signed [DATA_WIDTH-1:0]  a_out,
  output logic signed [DATA_WIDTH-1:0]  b_out,
  output logic signed [ACCUM_WIDTH-1:0] accum_out
);
```

Port Descriptions

Port	Dir	Connects To	Description
a_in[7:0]	in	a_in[i] or MAC[i][j-1].a_out	Left input. A value flows right through array.
b_in[7:0]	in	b_in[j] or MAC[i-1][j].b_out	Top input. B value flows down through array.
valid	in	feeder.valid (broadcast)	Single-cycle pulse. Accumulate a_in×b_in this cycle.
clear	in	controller.clear (broadcast)	Single-cycle pulse. Reset accumulator to zero.
a_out[7:0]	out	MAC[i][j+1].a_in	Registered a_in. Propagates A rightward every cycle.
b_out[7:0]	out	MAC[i+1][j].b_in	Registered b_in. Propagates B downward every cycle.
accum_out	out	systolic_array.results[i][j]	Combinational from accumulator FF. Valid 1 cycle after valid pulse.

RTL Behavior

```
always_ff @(posedge clk or negedge rst_n) begin
  if (!rst_n) begin
    a_out <= 0;  b_out <= 0;  accumulator <= 0;
  end else begin
    a_out <= a_in;          // always propagate every cycle
    b_out <= b_in;          // always propagate every cycle
    if (clear)              // clear takes priority over valid
      accumulator <= 0;
    else if (valid)
```

```
        accumulator <= accumulator + (a_in * b_in);
    end
end
assign accum_out = accumulator;  // combinational, 1-cycle latency
```

4.9 output_processor

Purpose

Selects one accumulator result per cycle from systolic_array using internal row/col counters and a MUX tree. Saturates 21-bit signed value to INT8 range [−128, +127]. Serializes ARRAY_SIZE² results one per cycle. Supports backpressure via ready_out. Asserts output_done after all results transferred.

Parameters

Parameter	Default	Description
ARRAY_SIZE	8	Controls result count (ARRAY_SIZE²)
ACCUM_WIDTH	21	Input accumulator width before saturation

Port Declaration

```
module output_processor #(
  parameter ARRAY_SIZE  = 8,
  parameter ACCUM_WIDTH = 21
)(
  input  logic                          clk,
  input  logic                          rst_n,
  input  logic signed [ACCUM_WIDTH-1:0]
    results [ARRAY_SIZE-1:0][ARRAY_SIZE-1:0],
  input  logic                          output_en,
  input  logic                          ready_out,
  output logic [7:0]                    data_out,
  output logic                          valid_out,
  output logic                          output_done
);
```

Port Descriptions

Port	Dir	Connects To	Description
results[i][j]	in	systolic_array.results[i][j]	All 64 accumulator values, continuously available. Read during OUTPUT state only.
output_en	in	controller.output_en	HIGH during OUTPUT state. Enables counter and MUX selection.
ready_out	in	host_interface.ready_out	Host can accept byte. LOW pauses output (backpressure).
data_out[7:0]	out	accelerator_core boundary	Saturated INT8 byte. Valid when valid_out=1 AND ready_out=1.

Port	Dir	Connects To	Description
valid_out	out	accelerator_core boundary	HIGH when result byte ready. Transfer = valid_out AND ready_out.
output_done	out	controller.output_done	Single-cycle pulse after ARRAY_SIZE² bytes transferred.

Internal Operation

- output_en=0: valid_out=0, counters held at 0, output_done=0.
- output_en=1: row_counter=0, col_counter=0, valid_out=1.
- Transfer (valid_out=1 AND ready_out=1): col_counter increments.
- col_counter wraps (0→ARRAY_SIZE-1): row_counter increments.
- Both counters wrap: output_done=1 (single-cycle pulse).
- Backpressure: ready_out=0 → valid_out stays HIGH, counters hold. Same byte repeated until accepted.
- Saturation: value > 127 → 8'h7F; value < -128 → 8'h80; otherwise value[7:0].

4.10 accelerator_axi_wrapper

Purpose

Reference AXI wrapper for SoC integration. Translates AXI4-Stream protocol to accelerator_core handshake interface. NOT synthesized for Chipathon tapeout. Verified through simulation only. TLAST maps to tile_done. TUSER[0] carries last_pass.

Parameters

Parameter	Default	Description
ARRAY_SIZE	8	Propagated to accelerator_core
ACCUM_WIDTH	21	Propagated to accelerator_core
TDATA_WIDTH	32	AXI4-Stream bus width. Width converter handles mismatch with 8-bit internal path.

Port Declaration

```
module accelerator_axi_wrapper #(
  parameter ARRAY_SIZE  = 8,
  parameter ACCUM_WIDTH = 21,
  parameter TDATA_WIDTH = 32
)(
  input  logic                    clk,
  input  logic                    rst_n,
  input  logic [TDATA_WIDTH-1:0]  s_axis_tdata,
  input  logic                    s_axis_tvalid,
  output logic                    s_axis_tready,
  input  logic                    s_axis_tlast,
  input  logic                    s_axis_tuser,
```

```
    output logic [TDATA_WIDTH-1:0]  m_axis_tdata,
    output logic                    m_axis_tvalid,
    input  logic                    m_axis_tready,
    output logic                    m_axis_tlast
);
```

AXI to Internal Signal Mapping

AXI Signal	Dir	Internal Signal	Notes
s_axis_tdata[7:0]	→	data_in	Width converter splits 32-bit into 4×8-bit bytes
s_axis_tvalid	→	valid_in	Direct mapping
s_axis_tready	←	ready_in	Direct mapping (ready_in is permission signal)
s_axis_tlast	→	tile_done	TLAST on 128th byte = tile_done single-cycle pulse
s_axis_tuser[0]	→	last_pass	DMA sets TUSER=1 on final tile
m_axis_tdata[7:0]	←	data_out	Width packer assembles bytes into TDATA
m_axis_tvalid	←	valid_out	Direct mapping
m_axis_tready	→	ready_out	Direct mapping
m_axis_tlast	←	output_done	Asserted on last result byte

Section 5 — Dataflow and Timing

5.1 SRAM Read Sequence

Each tile = 128 bytes: 64 bytes A tile + 64 bytes B tile. Feeder uses pipelined reads: presents addr_X at cycle N, Q valid at cycle N+1, staging captures at cycle N+2. This achieves 1 byte per cycle throughput after initial 2-cycle latency.

```
Cycle N:   feeder presents read_addr=X, read_en=1
Cycle N+1: Q = mem[X] valid on wire (~4.5ns after rising edge)
Cycle N+2: feeder staging FF captures Q at rising edge ✓
```

Per Inner Step k (16 cycles total)

```
Cycles 1–8:   present A column k: addr = row×ARRAY_SIZE+k, row=0→7
Cycles 9–16:  present B row k:    addr = 64+k×ARRAY_SIZE+col, col=0→7
Cycle 16:     valid=1 (SINGLE CYCLE PULSE)
Cycle 17:     skew buffer shifts, inner step k+1 begins
```

5.2 SRAM Address Formulas

Tile	Element	Formula	Example (k=2, row=3, col=5)
A tile	A[row][k]	addr = row × ARRAY_SIZE + k	addr = 3×8+2 = 26
B tile	B[k][col]	addr = ARRAY_SIZE² + k × ARRAY_SIZE + col	addr = 64+2×8+5 = 85

5.3 Array Input Pattern (4-Tile Example, Notation XYZ: X=tile, Y=row, Z=inner step)

Shift/Cycle	a_in[0]	a_in[1]	a_in[2]	...	a_in[7]	valid
0 / cy16	000	0	0	...	0	1
1 / cy32	001	010	0	...	0	1
7 / cy128	007	016	025	...	070	1
8 / cy144	100	017	026	...	071	1
31 / cy512	307	316	325	...	370	1
D1 / cy513	0	317	326	...	371	1
D7 / cy519	0	0	0	...	377	1
D8 / cy520	0	0	0	...	0	1
D14/ cy526	0	0	0	...	0	1
cy527	—	—	—	—	—	0 ← drain_done=1

5.4 Drain Phase Detail

Drain Cycles	Clock Cycles	valid	SRAM Read	Description
1–7	513–519	1 every cycle	No	Real data exits skew buffer. Row i finishes at drain cycle i. Zeros for completed rows.
8–14	520–526	1 every cycle	No	All zeros. Last real data propagates from array edges to MAC[7][7] (7 hops).
After 14	527	0	No	drain_done=1 pulse. All 64 MAC results valid. Controller → OUTPUT.

5.5 Swap and Ready_in Timing

```
Cycle 128: tile_done=1 (last byte written at rising edge 128)
           valid_in=1, ready_in=0, write_en=1 (=valid_in), write_addr=127

Cycle 129: swap=1 AND ready_in=1 (SAME CYCLE, controller FF output)
           write_en=0 (valid_in=0, host stopped after tile_done)
           write_addr=0 (reset for next tile)
           Memory not yet swapped (swap takes effect at edge 130)

Cycle 130: Memory swaps (SRAM_1 active, SRAM_0 write target)
           Feeder restarts (read_en=1, read_addr=0)
           AXI host: valid_in=1 ← first byte of next tile
                     write_en=1, SRAM_0 captures at edge 131 ✓
           USB host: still waiting (ms latency ≫40ns)
           ready_in stays HIGH until first valid transfer

Cycle 131: ready_in→0 (first transfer consumed permission)
           Host sends remaining bytes freely (ready_in=0)
```

5.6 Signal Behavior Summary

Signal	Type	Duration	Generated By
valid_in	Level	HIGH while host sending	Host
ready_in	Permission pulse	HIGH from swap until first transfer of new tile	Controller
tile_done	Pulse	1 cycle	Host
last_pass	Level	HIGH during final tile	Host
valid_out	Level	HIGH while chip sending results	output_processor
ready_out	Level	HIGH when host can receive	Host

Signal	Type	Duration	Generated By
valid (feeder→array)	Pulse	1 cycle per 16-cycle block + every cycle during drain	Feeder
write_en	Combinational	= valid_in (in LOAD states)	Controller
swap	Pulse	1 cycle (on tile_done, non-last)	Controller
clear	Pulse	1 cycle (after output_done)	Controller
output_en	Level	HIGH during OUTPUT state	Controller
drain_done	Pulse	1 cycle (after 14 drain cycles)	Feeder
output_done	Pulse	1 cycle (after ARRAY_SIZE² results)	output_processor

Section 6 — Memory Map and SRAM Layout

6.1 SRAM Organization (128 bytes per macro)

Address Range	Content	Storage Order	Access Formula
0 – 63	A tile (8×8 = 64 bytes)	Row-major	addr = row × ARRAY_SIZE + col
64 – 127	B tile (8×8 = 64 bytes)	Row-major	addr = 64 + row × ARRAY_SIZE + col

6.2 Ping-Pong State Table

State	SRAM_0	SRAM_1
INITIAL_LOAD	← host writes first tile	Idle
COMPUTE_AND_LOAD (after 1st swap)	Feeder reads →	← host writes next tile
COMPUTE_AND_LOAD (after 2nd swap)	← host writes next tile	Feeder reads →
OUTPUT	Idle (no read or write)	Idle (no read or write)
IDLE	Idle	Idle

6.3 Write Address Generation

Controller generates write_addr as a 7-bit registered counter. Increments on every cycle where valid_in=1 in LOAD states (write_en=valid_in). Resets to 0 on swap. Host writes A tile first (addresses 0–63) then B tile (addresses 64–127). Counter stalls when valid_in=0, ensuring no address is skipped during host stalls. Note: ready_in is tile-level permission only and does NOT control per-byte write_addr increments.

Section 7 — Pin Interface and Protocol

7.1 Physical Pin Assignment

Pin	Direction	Width	Signal Type	Description
clk	input	1	Continuous	System clock (25 MHz target)
rst_n	input	1	Level	Active low asynchronous reset
pad_data[7:0]	bidir	8	Level	Shared bus: tile data in OR result bytes out
pad_valid_in	input	1	Level	Host holds HIGH while sending tile bytes
pad_ready_in	output	1	Permission pulse	Held HIGH from swap until first byte of new tile. LOW during OUTPUT.
pad_tile_done	input	1	Single-cycle pulse	Host pulses after 128th byte transferred
pad_last_pass	input	1	Level	Host asserts before/during final tile
pad_valid_out	output	1	Level	Chip holds HIGH while sending result bytes
pad_ready_out	input	1	Level	Host holds HIGH when ready to receive bytes
Total	—	16 pins	—	6 spare pins available for debug or expansion

7.2 Handshake Mechanism

```
Byte write:      valid_in=1 (in LOAD state)  → one byte written to SRAM (write_en=valid_in)
Output transfer: valid_out=1 AND ready_out=1 → one byte transferred this cycle

write_en = valid_in (in LOAD states, combinational. NOT valid_in AND ready_in)
SRAM write: data captured at rising edge when write_en=1 ✓
```

7.3 Multi-Pass Protocol Sequence

```
After reset:
  Controller: ready_in=1 ← permission for first tile
  Host sees ready_in=1 → starts sending tile 1
  First transfer: ready_in→0 (permission consumed)

Pass 1 (last_pass=0):
  Host sends 128 bytes, asserts tile_done
  Controller: swap=1 AND ready_in=1 (SAME CYCLE) ← permission for tile 2
  Host waits for ready_in=1, then sends tile 2
```

```
Pass 2...N-1 (last_pass=0): repeat above

Pass N (last_pass=1):
  Host sends 128 bytes, asserts tile_done
  Controller: NO swap, NO ready_in pulse
  Feeder: drain phase (14 cycles) → drain_done
  Controller: output_en=1
  Chip: valid_out=1, sends 64 result bytes
  output_done → clear → ready_in=1 (new computation)
```

7.4 ready_in Behavior Detail

Cycle	Condition	ready_in	Explanation
Reset	Initial state	1	Permission for first tile granted after reset
1	First valid transfer	1→0	Permission consumed. ready_in goes LOW.
2–128	Host sending bytes	0	Host sends freely. No per-byte permission needed.
129	tile_done=1 (non-last)	1 (same as swap)	Permission for next tile granted simultaneously with swap.
130	First transfer of new tile	1→0	Permission consumed when first byte of new tile arrives.
OUTPUT	output_en=1	0	Not accepting new tile data during output phase.
After output_done	clear pulse	1	Permission for new computation granted.

7.5 Comparison with AXI4-Stream

Our Signal	AXI4-Stream Equivalent	Behavior
valid_in	TVALID	Level: HIGH when sender has valid data
ready_in	TREADY	Permission pulse: held HIGH until first byte received
tile_done	TLAST	Single-cycle pulse on 128th byte
last_pass	TUSER[0]	Sideband level signal alongside tile data
valid_out	TVALID (master)	Level: HIGH when result byte ready
ready_out	TREADY (master)	Level: backpressure from host

Section 8 — Area Estimates

8.1 PDK Cell Areas

Cell	Function	Area (µm²)
addf_1	Full Adder	72.4416
addh_1	Half Adder	39.5136
dffq_1	D Flip-Flop (no reset)	63.6608
dffrnq_1	D Flip-Flop (with reset)	74.6368
nand2_1	2-input NAND	10.9760
mux2_1	2-to-1 MUX	28.5376
mux4_1	4-to-1 MUX	70.2464

8.2 SRAM Macro and Timing (gf180mcu_fd_ip_sram__sram128x8m8wm1)

SRAM dimensions: Width 431.86 µm, Height 268.88 µm, Area 116,118.52 µm² per macro.
Timing parameters verified from Liberty file at typical corner (tt), 25°C, 5V:

Parameter	Value	Notes
min_period	5.88 ns	Supports up to ~170 MHz. Our 25 MHz target (40 ns) is well within spec.
CLK→Q delay	~4.5 ns	Q valid ~4.5 ns after rising edge. Stable 35.5 ns before next edge at 25 MHz.
Read latency	2 cycles	Pipelined: addr at cycle N, Q valid at N+1, staging captures at N+2.
Write latency	1 cycle	Data, addr, write_en stable before rising edge → captured at that edge.

8.3 Standard Cell Area Breakdown

Component	Cell Area (µm²)
64 MAC units	566,044.80
output_processor	32,249.68
Feeder	37,843.05
Controller	1,266.63
host_interface	632.22
Total standard cell area	638,036.38

8.4 MAC Unit Area Breakdown

Sub-component	Area (µm²)
8×8 Baugh-Wooley multiplier	4,693.33
21-bit accumulator adder	1,521.27
21-bit accumulator register (dffrnq_1)	1,567.37
8-bit A flow register	509.29
8-bit B flow register	509.29
Enable gating	43.90
Total per MAC unit	8,844.45
Total for 64 MAC units	566,044.80

8.5 Feeder Area Breakdown

Sub-component	FFs	Cell Area (µm²)
A staging registers (8×8-bit)	64	4,074.29
B staging registers (8×8-bit)	64	4,074.29
A skew buffer (28 stages × 8-bit)	224	14,260.02
B skew buffer (28 stages × 8-bit)	224	14,260.02
Read counter (7-bit, 0→127)	7	445.63
Phase tracker (1-bit, A vs B)	1	63.66
Inner step counter (3-bit, 0→7)	3	190.98
Drain counter (4-bit, 0→13)	4	254.64
Control logic (~20 × nand2_1)	—	219.52
Total Feeder	591	37,843.05

8.6 Controller Area Breakdown

Sub-component	Cell Area (µm²)
FSM state register (2-bit, 4 states)	127.32
write_addr counter (7-bit)	445.63
SRAM swap register (1-bit)	63.66
ready_in register (1-bit)	63.66
output_en register (1-bit)	63.66
last_pass_f register (1-bit)	63.66

Sub-component	Cell Area (µm²)
Next state + output logic (~40 × nand2_1)	439.04
Total Controller	1,266.63

8.7 output_processor Area Breakdown

Sub-component	Cell Area (µm²)
Output selection MUX (1-of-64, 21-bit)	30,978.66
Saturation clip logic (21-bit → 8-bit)	316.11
8-bit output register	509.29
Row counter (3-bit)	190.98
Column counter (3-bit)	190.98
output_done register (1-bit)	63.66
Total output_processor	32,249.68

8.8 host_interface Area Breakdown

Sub-component	Cell Area (µm²)
Bidir bus OE MUX (8 × mux2_1)	228.30
OE control logic (2 × nand2_1)	21.95
Input sync registers (4 × dffq_1)	254.64
Output registers (2 × dffq_1)	127.32
Total host_interface	632.22

8.9 Grand Total at Different Utilizations

	65% Utilization	70% Utilization	75% Utilization
Standard cells placed	981,594 µm²	911,481 µm²	850,715 µm²
Two sram128x8 (fixed)	232,237 µm²	232,237 µm²	232,237 µm²
Grand total	1,213,831 µm²	1,143,718 µm²	1,082,952 µm²
Slot A (1,248,806 µm²)	97.2% ✅	91.6% ✅	86.7% ✅

All three utilization targets fit within Slot A. 70% utilization is the recommended realistic target for this structured systolic array design.

8.10 Area Breakdown at 70% Utilization

Total Slot A: 1,248,806 µm²

Component	Cell Area (µm²)	Placed Area (µm²)	% of Slot A
64 MAC units	566,045	808,636	64.8%
SRAM_0 (sram128x8)	116,119 (fixed)	116,119	9.3%
SRAM_1 (sram128x8)	116,119 (fixed)	116,119	9.3%
output_processor	32,250	46,071	3.7%
Feeder	37,843	54,062	4.3%
Controller	1,267	1,810	0.1%
host_interface	632	903	0.1%
Total	870,275	1,143,720	91.6%

Note: SRAM macros are hard macros — placed area equals cell area (no utilization factor). Standard cells are placed at 70% utilization.

8.11 Accumulator Width Sensitivity Analysis

ACCUM_WIDTH affects MAC accumulator registers and the output selection MUX. Each additional bit increases total area by 15,554.56 µm² at 70% utilization.

Extra area per additional bit (at 70%):

MAC units: 64 × (addf_1 + dffrnq_1) = 9,413.02 ÷ 0.70 = 13,447.17 µm²

Output MUX: 21 × mux4_1 = 1,475.17 ÷ 0.70 = 2,107.39 µm²

Total per bit: 15,554.56 µm²

Baseline (ACCUM_WIDTH=21, 70%): 1,143,718 µm²

Extra Bits	ACCUM_WIDTH	Max K	Application Coverage	Total Extra Area	New Total	Slot A %
0	21	32	ECG, basic biosignals	0 µm²	1,143,718 µm²	91.6% ✅
1	22	64	+ Gesture, anomaly, keyword	15,555 µm²	1,159,273 µm²	92.8% ✅
2	23	128	+ Tiny image classification	31,109 µm²	1,174,827 µm²	94.1% ✅
3	24	256	+ Small speech recognition	46,664 µm²	1,190,382 µm²	95.3% ✅
4	25	512	+ Medium CNN	62,218 µm²	1,205,936 µm²	96.6% ✅
5	26	1,024	+ Large CNN, ResNet	77,773 µm²	1,221,491 µm²	97.8% ✅
6	27	2,048	+ BERT-small	93,327 µm²	1,237,045 µm²	99.1% ✅

Extra Bits	ACCUM_WIDTH	Max K	Application Coverage	Total Extra Area	New Total	Slot A %
7	28	4,096	+ GPT-2 small	108,882 µm²	1,252,600 µm²	100.3% ⚠
8	29	8,192	+ BERT-base	124,436 µm²	1,268,154 µm²	101.5% ⚠
9	30	16,384	+ GPT-2 medium	139,991 µm²	1,283,709 µm²	102.8% ⚠
10	31	32,768	+ GPT-3 small	155,546 µm²	1,299,263 µm²	104.0% ⚠

- Default ACCUM_WIDTH=21 (K≤32) fits at 91.6% ✅
- All configurations up to ACCUM_WIDTH=27 (K≤2,048) fit within Slot A ✅
- ACCUM_WIDTH=28 and above exceeds Slot A at 70% utilization ⚠
- ACCUM_WIDTH is a synthesis-time parameter — no RTL changes required ✅

8.12 Important Caveats

- Slot A area taken as 1,248,806 µm² (= 2235 × 2235 µm total die ÷ 4) per Chipathon 2026 padframe proposal.
- Cell counts are based on textbook decompositions. Yosys+ABC synthesis may reduce area 10–20% using compound cells (AOI/OAI).
- Utilization of 70% is based on the regular, structured nature of systolic array designs. Confirmed only after LibreLane place-and-route.
- FSM and control logic estimates (~40 gates) are approximations. Actual synthesis may vary.
- SRAM timing verified from Liberty file (tt, 25°C, 5V): min_period = 5.88 ns, CLK→Q ≈ 4.5 ns, read latency = 2 cycles (pipelined).
- The 8.4% free margin at 70% utilization provides buffer for routing overhead and underestimation in cell counts.
- Silicon validation required for final confirmation.

Section 9 — Parameters

9.1 Design Parameters

Parameter	Default	Valid Values	Constraint	Used In
ARRAY_SIZE	8	{2,4,8}	$2 \times N^2 \leq 128$	accelerator_top, accelerator_core, feeder, controller, systolic_array, output_processor
DATA_WIDTH	8	8 only	Fixed by sram128x8	systolic_array, mac_unit (localparam elsewhere)
ACCUM_WIDTH	21	≥21	For K≤32	accelerator_top, accelerator_core, systolic_array, mac_unit, output_processor
TDATA_WIDTH	32	{8,32,64}	AXI bus width	accelerator_axi_wrapper only

9.2 Derived Localparams

```
localparam DATA_WIDTH   = 8;
localparam SRAM_DEPTH   = 128;
localparam ADDR_WIDTH   = 7;                              // $clog2(128)
localparam TILE_BYTES   = 2 * ARRAY_SIZE * ARRAY_SIZE;    // 128 for N=8
localparam RESULT_BYTES = ARRAY_SIZE * ARRAY_SIZE;        // 64 for N=8
localparam DRAIN_CYCLES = 2 * (ARRAY_SIZE - 1);           // 14 for N=8
```

9.3 ACCUM_WIDTH Derivation

```
Maximum INT8 × INT8: (−128)×(−128) = 16,384 = 2^14 → 15 bits unsigned

Maximum accumulated value for K=32 passes:
  32 × 16,384 = 524,288 = 2^19
  2^19 requires 20 bits unsigned (bit 19 must be set)
  + 1 sign bit = 21 bits signed ✓

21-bit signed range: -1,048,576 to +1,048,575
Max value 524,288 ≤ 1,048,575 ✓ safely within range
```

Section 10 — AXI Wrapper (IP Reusability)

10.1 Purpose

The accelerator_axi_wrapper provides plug-and-play AXI4-Stream compatibility for SoC integration. Provided as a GitHub reference implementation. NOT synthesized for Chipathon tapeout. Verified through simulation only.

10.2 Typical SoC Integration Workflow

```
1. CPU configures DMA: N tiles, TUSER=0 for non-final, TUSER=1 for last
2. DMA streams tile 1: TUSER=0, TLAST=1 on byte 128
3. Accelerator computes tile 1, loads tile 2 simultaneously (ping-pong)
4. DMA streams tile N: TUSER=1, TLAST=1 on byte 128
5. Feeder drains (14 cycles) → drain_done → OUTPUT phase
6. Accelerator streams 64 result bytes: TLAST=1 on last byte
7. DMA receives results → writes to RAM → CPU reads
```

10.3 Repository Structure

```
rtl/         mac_unit.sv, systolic_array.sv, feeder.sv,
             memory.sv, controller.sv, output_processor.sv, accelerator_core.sv
axi_wrapper/ axi4_stream_slave.sv, axi4_stream_master.sv, accelerator_axi_wrapper.sv
chipathon/   host_interface.sv, accelerator_top.sv
tb/          tb_*.sv for all modules
docs/        architecture_spec.md, port_specification.md, axi_integration_guide.md
```

Section 11 — Testbench Plan

11.1 Simulation Tools

- Primary: cocotb (Python-based verification framework)
- Simulators: Icarus Verilog, Verilator
- Python generates test vectors, computes expected results via numpy, compares and reports PASS/FAIL

11.2 Unit Testbenches (All Modules)

Testbench	Module	Key Tests
tb_mac_unit	mac_unit	Single/multiple accumulations, clear priority, signed multiply (-128×-128), a_out/b_out propagation every cycle, 1-cycle latency verification
tb_systolic_array	systolic_array	8×8 matrix multiply with correct skewed inputs, all 64 results vs numpy, clear resets all accumulators
tb_feeder	feeder	Pipelined SRAM reads (2-cycle latency), skew buffer pattern vs expected table, valid single-cycle pulse, warmup zeros, drain phase (14 cycles), drain_done timing, swap_pulse restart without reset, clear full reset
tb_memory	memory	Write then read same address, ping-pong swap, simultaneous read+write (different SRAMs), write_en gating, SRAM 2-cycle read latency
tb_controller	controller	All FSM transitions, write_addr counter, write_en=valid_in (not AND ready_in), swap AND ready_in same cycle, stall behavior, ready_in permission pulse held until first byte
tb_output_processor	output_processor	Saturation clipping (±127/±128 boundaries), serialization order, backpressure (ready_out=0), output_done timing, 64 bytes total
tb_host_interface	host_interface	Bidirectional pad_data OE control, all signal pass-throughs, ready_in permission pulse behavior
tb_accelerator_core	accelerator_core	End-to-end: 8×8 single pass, 16×16 two pass, 32×32 four pass, back-to-back tiles, stall/resume, ready_out backpressure, multiple consecutive computations
tb_accelerator_top	accelerator_top	Structural wiring verification via pad signals
tb_axi_wrapper	accelerator_axi_wrapper	AXI4-Stream protocol compliance, TDATA width conversion, TLAST→tile_done,

Testbench	Module	Key Tests
		TUSER→last_pass, TREADY permission behavior

11.3 Physical Chip Testing

- Laptop Python → USB/FTDI FT2232H → PCB → bond wires → GF180MCU chip
- Python generates random A, B matrices; computes expected C = A×B via numpy
- Drives physical pins per protocol: ready_in pulse → send 128 bytes → tile_done → repeat
- Reads 64 result bytes, compares vs expected, reports PASS/FAIL
- All submodules indirectly verified: correct chip output requires all internal modules working correctly

11.4 Verification Checklist

Test	Simulation	Physical Chip	Notes
mac_unit correctness	✅	✅ (indirect)	Direct in sim; verified via chip output
systolic_array correctness	✅	✅ (indirect)	Direct in sim; verified via chip output
feeder timing and skew	✅	✅ (indirect)	Verified against expected table in sim
memory ping-pong	✅	✅ (indirect)	Verified in isolation in sim
controller FSM	✅	✅ (indirect)	All states and transitions in sim
output_processor saturation	✅	✅ (indirect)	Boundary cases in sim
accelerator_core 8×8	✅	✅	Both simulation and silicon
accelerator_core 16×16	✅	✅	Both simulation and silicon
accelerator_core 32×32	✅	✅	Both simulation and silicon
Stall and resume	✅	✅	valid_in toggling during transfer
Backpressure (ready_out)	✅	✅	Output stall behavior
Back-to-back tiling	✅	✅	Ping-pong and feeder continuity
ready_in permission pulse	✅	✅	Correct timing of permission grant
AXI wrapper compliance	✅	—	Simulation only, not on chip